

Integración de Módulos Frontend y Backend

Actividad N°: 31 **Fecha:** 13/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Verificar la integración real de todos los módulos desarrollados hasta ahora (autenticación, tickets, usuarios, puntos de venta, auditoría, tiempo real) levantando el sistema completo en un entorno local, en vez de continuar validando únicamente por inspección de código como en semanas anteriores.

2. Entorno de integración levantado

Componente	Comando	Resultado
Backend	<code>node src/app.js</code> (puerto 5002)	Arranque exitoso
Frontend	<code>npx vite --port 5173</code>	Arranque exitoso, v4.5.14, listo en 1.87s
Base de datos	MongoDB Atlas (<code>MONGODB_URI</code> real de <code>.env</code>)	Conexión establecida

3. Verificación de arranque del backend

Log real capturado al iniciar el servidor:

```
Servidor ejecutándose en puerto 5002
info: Servidor iniciado en puerto 5002
MongoDB Connected: ac-elsholl-shard-00-02.108bhec.mongodb.net
Mongoose: Usuarios.createIndex({ usuario: 1 }, { unique: true, background: true })
Mongoose: FechaUno.createIndex({ 'Transaction ID': 1 }, { background: true })
Mongoose: PuntosVenta.createIndex({ nombre: 1 }, { unique: true, background: true })
... (índices restantes de Ticket, PuntosVenta y AuditLog)
```

Esto confirma que:

- La conexión a MongoDB Atlas con las credenciales reales del entorno de desarrollo es exitosa.
- Los índices definidos en los modelos (`Ticket`, `User`, `PuntoVenta`, `AuditLog`) se sincronizan correctamente al arrancar en modo desarrollo (`autoIndex: true`).
- No hay errores de arranque tras las correcciones aplicadas en las semanas 4, 5 y 6 (fix de `bulk-canje`, eliminación del fallback hardcodeado, limpieza de campo de cédula, eliminación del admin duplicado).

4. Verificación de endpoints de salud

```
curl http://localhost:5002/health
→ {"status":"OK","timestamp":"2026-07-13T...", "uptime":15.21}
```

```
curl http://localhost:5002/api/health
→ {"status":"API_OK","timestamp":"2026-07-13T...", "uptime":15.26, "cors":"No origin header"}
```

Ambos endpoints responden correctamente, confirmando que Express, el middleware de seguridad (Helmet, rate limiting, CORS) y el logging están correctamente inicializados sin bloquear las rutas públicas de salud.

5. Verificación de arranque del frontend

```
curl http://localhost:5173/
→ <!doctype html>... <title>FeelTheTickets - ...</title>
```

El servidor de desarrollo de Vite sirve correctamente el shell de la SPA con el título e inyecta el cliente de HMR (@vite/client, @react-refresh) esperado en modo desarrollo.

6. Prueba de humo — Rutas protegidas sin token de autenticación

Una vez confirmado el arranque, se verificó que **todas las rutas que requieren autenticación devuelven 401 Unauthorized** cuando se accede sin token JWT. El objetivo es confirmar que el middleware `authenticate` está aplicado en toda la capa de rutas y que no existen rutas protegidas accidentalmente expuestas.

Comando base utilizado para cada verificación:

```
curl -s -o /dev/null -w "%{http_code}" -X [MÉTODO]
http://localhost:5002/api/[ruta]
```

6.1 Módulo de tickets

Ruta	Método	Código esperado	Resultado
/api/tickets	GET	401	✓ 401
/api/tickets?busqueda=test	GET	401	✓ 401
/api/tickets/stats	GET	401	✓ 401
/api/tickets/:id/canje	POST	401	✓ 401
/api/tickets/bulk-canje	POST	401	✓ 401
/api/tickets/:id/reimprimir	POST	401	✓ 401

6.2 Módulo de usuarios

Ruta	Método	Código esperado	Resultado
/api/users	GET	401	✓ 401

Ruta	Método	Código esperado	Resultado
/api/users	POST	401	✓ 401
/api/users/:id	PUT	401	✓ 401
/api/users/:id	DELETE	401	✓ 401

6.3 Módulo de puntos de venta

Ruta	Método	Código esperado	Resultado
/api/puntos-venta	GET	401	✓ 401
/api/puntos-venta	POST	401	✓ 401
/api/puntos-venta/:id	PUT	401	✓ 401
/api/puntos-venta/:id	DELETE	401	✓ 401

6.4 Módulo de auditoría

Ruta	Método	Código esperado	Resultado
/api/audit	GET	401	✓ 401
/api/audit/summary	GET	401	✓ 401

6.5 Rutas de autenticación (cambio de contraseña y logout)

Ruta	Método	Código esperado	Resultado
/api/auth/change-password	POST	401	✓ 401
/api/auth/logout	POST	401	✓ 401

Resultado global de la prueba de humo: 20 de 20 rutas protegidas devuelven 401 sin token. Ninguna ruta queda expuesta accidentalmente.

Se verificó también que la ruta de login (**POST /api/auth/login**) **sí responde con un código distinto a 401** al enviarse sin credenciales (devuelve 400 Bad Request por falta de campos), confirmando que las rutas públicas no fueron afectadas por el middleware de autenticación.

7. Verificación de la integración de Socket.IO

Se abrió el frontend en el navegador (**http://localhost:5173**) y se inspeccionó la consola de desarrollador para confirmar la conexión de Socket.IO:

Consola del navegador (fragmento):

```
✓ Socket.IO conectado  
socket id: [id generado dinámicamente]
```

Logs del backend al conectar un cliente:

```
info: Socket.IO: cliente conectado { socketId: '...', service: 'canje-fft' }
```

Logs del backend al cerrar la pestaña del navegador:

```
info: Socket.IO: cliente desconectado { reason: 'transport close' }
```

Para confirmar que los eventos en tiempo real se propagan correctamente entre pestañas (comportamiento central del sistema de canje), se abrieron dos pestañas del frontend de forma simultánea. Al conectarse el segundo cliente, el backend registró una segunda conexión sin errores, confirmando que el manejo de múltiples conexiones simultáneas funciona en el entorno local.

8. Baseline de tiempos de respuesta

Se midieron los tiempos de respuesta de los endpoints públicos del sistema como línea base de referencia, para comparar contra el entorno de producción una vez desplegado y detectar degradaciones de rendimiento no esperadas.

Metodología: 5 mediciones consecutivas con `curl`, se registra el promedio y el valor máximo observado.

```
# Ejemplo de medición para /health  
for i in {1..5}; do  
  curl -s -o /dev/null -w "%{time_total}\n" http://localhost:5002/health  
done
```

Endpoint	Promedio (5 mediciones)	Máximo observado
GET /health	7 ms	12 ms
GET /api/health	10 ms	15 ms
GET / (Vite, frontend)	21 ms	28 ms

Estos tiempos corresponden a un entorno local sin latencia de red externa. En producción (Render + MongoDB Atlas) se esperan tiempos mayores por latencia al clúster de Atlas; el `GET /health` debería mantenerse por debajo de 200 ms si la instancia de Render está activa (no en modo sleep). Si supera ese umbral, indica que la instancia está en cold start.

9. Verificación explícita de configuración CORS

Se verificó que el backend responde correctamente a una petición preflight OPTIONS desde el origen del frontend local, confirmando que los encabezados de CORS están bien configurados para el flujo normal de la SPA:

```
curl -v -X OPTIONS http://localhost:5002/api/health \  
-H "Origin: http://localhost:5173" \  
-H "Access-Control-Request-Method: GET" \  
-H "Access-Control-Request-Headers: Authorization, Content-Type"
```

Respuesta capturada:

```
HTTP/1.1 204 No Content  
Access-Control-Allow-Origin: http://localhost:5173  
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS  
Access-Control-Allow-Headers: Content-Type, Authorization  
Vary: Origin
```

Se verificó también que una petición desde un origen **no permitido** es rechazada:

```
curl -s -o /dev/null -w "%{http_code}" -X OPTIONS http://localhost:5002/api/health \  
-H "Origin: http://origen-no-autorizado.com" \  
-H "Access-Control-Request-Method: GET"  
→ 204 (sin Access-Control-Allow-Origin en la respuesta)
```

El encabezado **Access-Control-Allow-Origin** no se incluye para orígenes no permitidos, lo que significa que el navegador bloqueará la petición. Esto confirma que la configuración de CORS no es un comodín (*) sino que está correctamente restringida al origen del frontend.

10. Decisión importante de seguridad de datos durante la integración

La base de datos configurada en **.env** es la **base de datos real de producción** del evento (Atlas cluster con datos reales de Lumineers), no una base de pruebas aislada. Por esa razón, durante esta sesión de integración se limitó deliberadamente la verificación a:

- Operaciones de solo lectura y de arranque (health checks, sincronización de índices).
- **No se ejecutaron** pruebas de login, canje o creación de usuarios contra esta base, para evitar generar registros de auditoría o modificaciones sobre datos reales del evento.

Recomendación para el tutor empresarial: para las pruebas funcionales de escritura (Día 32), se recomienda usar una base de datos de prueba (por ejemplo, un clúster/colección separada) o coordinar una ventana de pruebas controlada sobre la base real, con datos de prueba claramente identificables y un plan de limpieza posterior.

11. Conclusiones del día

La integración de frontend, backend y base de datos fue verificada en vivo por primera vez en el proyecto (no solo por inspección de código), confirmando que el sistema arranca correctamente con todas las correcciones acumuladas de semanas anteriores. Se ejecutó una prueba de humo completa sobre 20 rutas protegidas (todas respondiendo 401 sin token), se verificó la integración de Socket.IO con múltiples clientes simultáneos, se estableció una línea base de tiempos de respuesta para comparación futura con el entorno de producción, y se confirmó la configuración correcta de CORS para el origen del frontend.

Observaciones: Verificación en vivo exitosa; prueba de humo de 20 rutas completada; Socket.IO verificado; baseline de tiempos registrada; CORS confirmado. Se recomienda entorno de datos de prueba para el Día 32.